RESULTS

Phase-3 CPU Speech-To-Text (Whisper) — end-to-end proof (§11.4.108 / §11.4.5 / §11.4.69 / §11.4.107)

Date: 2026-07-07 · Track (`T1/feature/helixllm-full-extension`) · Capability code: `submodules/helix_llm/container/{Containerfile.whisper,whisper_stt_server.py}`

Verdict (honest, §11.4.6)

CAPABILITY PROVEN. A real CPU Speech-To-Text service booted via the **containers submodule orchestrator** (§11.4.76, rootless podman §11.4.161, `--build` from the HelixLLM submodule's own Containerfile, NO GPU) transcribed real synthesized audio into real text. The §11.4.108 runtime signature is **GREEN-OK for both distinct utterances**:

```
[RUNTIME-SIGNATURE(fox)]        PASS transcript="The quick brown fox dumps
                                     normalized="the quick brown fox dump
[RUNTIME-SIGNATURE(helloworld)] PASS transcript="Hello World 1, 2, 3!"
                                     normalized="hello world 1 2 3" missi
[DETERMINISM]                   PASS: normalized transcript identical acro
[SELF-VALIDATION]               PASS: analyzer PASSes both golden-good fix
```

Both transcripts genuinely recover the KNOWN text the harness itself synthesized with `espeak-ng` (deterministic TTS render of a literal string) — an unfakeable proof: the harness generated the audio from known text, and the running container had to recover that text from the waveform alone. Evidence: `11_green_proof_fox.txt`, `11_green_proof_helloworld.txt`, `13_determinism.txt`, `green_fox_{1,2}.json`, `green_helloworld_1.json`.

Engine choice (§11.4.150 deep multi-angle research)

Chosen: faster-whisper (CTranslate2), CPU, `int8`, model `base`.

Two research angles, both dated and cited:

1. **Existing deep-research design doc** (already in this repo, `docs/research/07.2026/ 07_stt_ocr_whisper_tesseract/`

`07_stt_ocr_whisper_tesseract.md`, sources verified 2026-07-06): *"whisper.cpp vs faster-whisper is a speed/platform choice, not an accuracy choice — all run identical weights [builderai.tools, codersera.com]. faster-whisper's CTranslate2 + int8 is the VRAM/speed winner on NVIDIA; whisper.cpp wins on CPU/Mac."* That doc's own executive recommendation for **STT (batch, multilingual)** is faster-whisper large-v3, with whisper.cpp named as the CPU/Mac runner-up — i.e. faster-whisper was ALREADY the primary pick; whisper.cpp is preferred only when the *host* (not the model) is Apple Silicon/CPU-constrained in a way that favours a native C++ binary over a Python process.

2. **Fresh confirming search, 2026-07-07** (`whisper.cpp server ggml base.en Docker CPU HTTP transcription 2026`; `faster-whisper CTranslate2 CPU int8 base model benchmark accuracy 2026`):
   - whisper.cpp ships an official `whisper-server` HTTP binary + Docker CUDA/CPU images and GGML `base.en` (142 MB) / quantized (57 MB, Q5_1) models — mature, but requires a *separate* GGML model-conversion pipeline distinct from the HF/CTranslate2 model format. (github.com/ggml-org/whisper.cpp, DeepWiki HTTP Server)
   - faster-whisper/CTranslate2 int8 on CPU: *"2x faster [than reference Whisper] on CPU … same accuracy … inference time typically drops to one-fourth … memory usage also decreases significantly"* with only a documented, small degradation on *aggressively* quantized inputs, not int8-CPU specifically. (localaimaster.com/blog/faster-whisper-guide, promptquorum.com/…local-whisper-stt-comparison-2026, github.com/AIXerum/faster-whisper)

Decision (§11.4.74 extend-don't-reimplement): this exact engine/model/compute-type combination (`faster-whisper==1.1.1`, CTranslate2 `4.8.1`, `model=base`, `device=cpu`, `compute_type=int8`, with a Silero-VAD + `no_speech_prob≥0.6` silence-hallucination guard) was **already implemented and proven working on this exact host** in a prior parallel-stream artifact, `docs/qa/p3_whisper_stt/` (commit `950b2fa3`) — real transcription, a documented + measured silence-hallucination guard, 3/3 deterministic runs. Re-deriving a whisper.cpp/GGML path here would add a second model format and a second serving binary for **no CPU-accuracy benefit** (same weights, same WER) and would violate §11.4.74/§11.4.6 (don't re-solve an already-solved, already-evidenced problem). This proof **reuses that proven engine choice**, adapts it into `submodules/helix_llm/container/` as a HelixLLM-owned capability (Containerfile + FastAPI server, §11.4.74 code lives in the submodule, not duplicated into evidence), rewires it to port **18437** (coder=18434, embeddings=18435, translation=18436 are taken), and re-proves it end-to-end through the containers-submodule orchestrator with a fresh two-utterance fixture set and a stronger self-validated analyzer.

Runtime signature

```
### GREEN runtime signature (§11.4.108) fixture=fox
[RUNTIME-SIGNATURE(fox)] PASS transcript="The quick brown fox dumps over t
                          normalized="the quick brown fox dumps over the l
signature_exit=0
GREEN-OK(fox)

### GREEN runtime signature (§11.4.108) fixture=helloworld
[RUNTIME-SIGNATURE(helloworld)] PASS transcript="Hello World 1, 2, 3!"
                                 normalized="hello world 1 2 3" missing=[]
signature_exit=0
GREEN-OK(helloworld)

### determinism (fox x2)
[DETERMINISM] PASS: normalized transcript identical across two identical r
determinism_exit=0
```

Runtime metadata from the served container (real HTTP 200 responses, `green_fox_1.json`): `language=en`, `max_no_speech_prob=0.0197` (well under the calibrated `0.6` hallucination floor — i.e. the model reported high confidence this is real speech, not silence/noise).

Honest engine-behavior note (§11.4.6 — normalization, not a defect)

Two real, expected ASR behaviors surfaced and are both handled honestly, not papered over:

1. **“jumps” → “dumps”** — the model mis-heard one word in the fox utterance (a real, minor ASR imperfection on synthetic espeak-ng TTS audio, which has a noticeably more robotic/lower-fidelity timbre than natural human speech). The runtime signature does **not** require “jumps” as a key content word (only “quick”, “brown”, “fox”, “lazy”, “dog” — the load-bearing content that proves real recovery of the known sentence); “dumps” is captured verbatim in the evidence file, not hidden.
2. **“one two three” → “1, 2, 3”** — Whisper’s documented text-normalization renders small spoken numbers as digits (the same behavior the prior proven artifact recorded: *“Whisper normalizes ‘forty two’→‘42’ (allowed)”*, `docs/qa/p3_whisper_stt/README.md`). The analyzer’s word-match was corrected to canonicalize digit/number-word pairs (`canonToken` in `harness/main.go`) so `"1"` and `"one"` compare equal — an honest reconciliation of the assertion with a real, evidenced engine behavior (§11.4.120), never a weakening: the

check still requires the literal content words `hello`/`world`/`one`/`two`/`three` (or their digit form) to be genuinely present in the transcript, and still correctly FAILs every golden-bad fixture below.

Analyzer is non-bluff (§11.4.107(10) / §11.4.115)

Proven to genuinely discriminate — it does NOT rubber-stamp:

- **RED baseline** (`10_red_baseline.txt`, §11.4.115): a canned **empty-text** stub AND a canned **wrong-content** stub, run through the IDENTICAL `analyze` subcommand the GREEN lane uses (no separate red-only code path), both correctly **FAIL** before the container ever boots. Defect reproduced.
- **Golden-BAD fixtures** (`12_self_validation.txt`) all correctly **FAIL**:
  - **real HTTP transcription of 3s digital silence** (`silence_response.json`) — the served container returned `text=""` for real (VAD dropped the non-speech region before the decoder could hallucinate) → correctly FAILs the fox signature.
  - **real HTTP transcription of 3s white noise** (`noise_response.json`) — likewise `text=""` for real → correctly FAILs.
  - **wrong-content**: the REAL "helloworld" transcript analyzed against the "fox" fixture's expected words → correctly FAILs (proves the analyzer rejects genuine, well-formed, non-empty — but WRONG — speech, not just empty/garbage input).
  - **empty transcript** (in-memory) → correctly FAILs.
- **Both real GREEN inputs** (fox, helloworld) correctly **PASS** against their own fixture (above).

Reproduce

```
cd docs/qa/phase3_whisper_stt_20260707/harness && ./run_proof.sh
```

Boots `helixllm-stt` via the containers submodule orchestrator (`--build` from `submodules/helix_llm/container/Containerfile.whisper`), synthesizes the two known utterances + silence + white-noise with `espeak-ng`/`ffmpeg`, POSTs each to `/v1/audio/transcriptions`, asserts the runtime signature + determinism + self-validation, tears the container down single-owner (§11.4.119), leaves `helixllm-coder` (and the sibling embeddings/translation capability containers) untouched. `HEALTH_TIMEOUT`/`STT_HOST_PORT`/`WHISPER_MODEL`/`WHISPER_COMPUTE_TYPE` env vars tune the run. `harness/phase3whisper.bin` and the synthesized `*.wav` fixtures are build artifacts (gitignored §11.4.30; regenerate via `go build .` and the `synth`/`synth-silence`/`synth-noise` subcommands).

Single-owner cleanup (§11.4.119) + transient host-fork note (§11.4.6)

The first `boot-down` attempt in this run hit a **transient host resource-exhaustion error** (`BlockingIOError: [Errno 11] Resource temporarily unavailable` inside `podman-compose`'s asyncio subprocess fork) — this host was concurrently running other parallel `§11.4.103` tracks at the time (confirmed: a sibling `phase3translatenllb_primary_nllb-shim_1` container was mid-boot on this same host during teardown). Retried `boot-down` 3 seconds later: `DOWN-OK`, container fully removed. Captured in `29_teardown.txt` (the failed attempt) + `29c_teardown_retry.txt` (the successful retry) + `29b_post_teardown.txt` (final state). Post-teardown: `helixllm-coder` still `Up`, port `18437` free, no `phase3whisperstt_cpu_*` container remains. Per §11.4.174, only processes/containers positively identified as ours (`phase3whisperstt_cpu_*`) were targeted at any point — the sibling translation container was left untouched throughout.

Composition

§11.4.76 (containers submodule) · §11.4.161 (rootless) · §11.4.74 (extend-don't-reimplement — reused the proven `docs/qa/p3_whisper_stt/` engine/model/guard choice) · §11.4.108 (runtime signature) · §11.4.107(10) (self-validated analyzer, real silence/noise negative HTTP probes) · §11.4.115 (RED-first, identical-codepath stub baseline) · §11.4.119 (single-owner teardown, retried past a transient host-fork error) · §11.4.150 (deep multi-angle engine-choice research) · §11.4.6 (honest ASR-normalization reconciliation, honest transient-failure retry) · §11.4.174 (process/container ownership verified before touching anything).

Sources verified 2026-07-07 (+ 2026-07-06 carry-forward)

1. `docs/research/07.2026/07_stt_ocr_whisper_tesseract/07_stt_ocr_whisper_tesseract.md` — deep multi-angle STT/OCR research, sources verified 2026-07-06 (21 external citations; see that file's own "Sources verified" footer).
2. github.com/ggml-org/whisper.cpp — *Port of OpenAI's Whisper model in C/C++, `whisper-server` HTTP binary, GGML `base.en`/quantized models* — https://github.com/ggml-org/whisper.cpp (accessed 2026-07-07).
3. DeepWiki — *whisper.cpp HTTP Server* — https://deepwiki.com/ggml-org/whisper.cpp/3.2-http-server (accessed 2026-07-07).

4. Local AI Master — *Faster-Whisper Setup Guide (2026): 4x Faster Local Speech-to-Text* — https://localaimaster.com/blog/faster-whisper-guide (accessed 2026-07-07).
5. PromptQuorum — *Whisper.cpp vs faster-whisper 2026: Local STT Benchmarks, Setup & GPU Acceleration* — https://www.promptquorum.com/power-local-llm/local-whisper-stt-comparison-2026 (accessed 2026-07-07).
6. github.com/AIXerum/faster-whisper — *CTranslate2 reimplementation, up to 4x faster on GPU / 2x on CPU, 8-bit quantization on both CPU and GPU* — https://github.com/AIXerum/faster-whisper (accessed 2026-07-07).

Negative findings / gaps (§11.4.99(B))

- Precise word-level accuracy of `base` vs `base.en`/larger models on non-English or noisy field audio was not re-measured here (out of scope — CPU `base` model per task); the design doc's own note stands: re-calibrate on the target corpus before wiring into a shipped HelixQA gate.
- whisper.cpp's CPU throughput was not directly benchmarked against faster-whisper on this exact host (no primary/reproducible harness found for that specific pairing); the decision rests on reuse-of-proven-artifact (§11.4.74) rather than a fresh head-to-head CPU benchmark, which is an honestly-stated limitation of this proof.

Review follow-ups (independent review §11.4.142 — VERDICT GO)

Independent adversarial review returned **GO** (0 blocking findings). Two Minor findings, disclosed here per §11.4.6:

1. **silence/noise responses byte-identical (honest note).** `silence_response.json` and `noise_response.json` are byte-for-byte identical (incl. `language_probability` ≈ 0.5679) despite two distinct HTTP calls on different WAVs. This is genuine faster-whisper behaviour on non-speech input: both WAVs contain no speech, so the VAD yields an empty transcript and the language-ID head falls back to a similar default-English confidence on degenerate input. It does NOT affect correctness — the analyzer inspects only `text`, which is independently `""` for both, so both golden-bad fixtures correctly FAIL.
2. **Analyzer is recall-only** (requires all expected words present; no ceiling on extra/hallucinated content). Not exploitable for the tested fixtures (they share no expected words); recorded as a design limitation for future overlapping-vocabulary fixtures.